

1.1 Exploratory Data Analysis (EDA)

- 1.1.1 Types of Exploratory Data Analysis
- 1.1.2 Univariate Analysis
- 1.1.3 Bivariate Analysis
- 1.1.4 Multivariate Analysis
- 1.1.5 Handling Missing Data and Outliers

1.2 Understanding the Data

- 1.2.1 Quantitative Data : Discrete and Continuous
- 1.2.2 Qualitative Data : Non-numerical (Normal and Ordinal)

1.3 Spread of Data

- 1.3.1 Normal Distribution
- 1.3.2 Skewed Distribution
- 1.3.3 Skewness and Kurtosis

1.1 Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) is the *first systematic and thoughtful examination* of a dataset performed before any formal modeling or hypothesis testing. It aims to help a data analyst, researcher, or machine-learning practitioner understand the **nature, structure, and quality** of the data, discover **patterns and trends**, and detect **anomalies, errors, or inconsistencies** that may affect subsequent analysis. It blends **descriptive statistics, visualization, and data-cleaning** to answer: *What do we have? Is it usable? What signals and relationships exist?*

EDA acts as the **bridge between raw data and modeling**. Instead of jumping straight into algorithms, analysts first explore the data visually and statistically to build intuition about what it contains and how it behaves.

Purpose of EDA

EDA answers a few essential questions:

1. “What do we have?”

- Identifies the **type** of data available (numerical, categorical, textual, temporal).
- Determines **dimensions**—number of rows, columns, and variables.
- Understands the **source and meaning** of each feature through metadata or data dictionaries.

2. “Is it usable?”

- Evaluates **data quality**—presence of missing values, outliers, duplicate records, and inconsistent formats.
- Checks **validity** of entries (e.g., negative ages, impossible timestamps).
- Ensures **data integrity**—correct data types, consistent scales, and uniform measurement units.

3. “What signals and relationships exist?”

- Reveals **distributions** of variables (central tendency, spread, skewness).
- Explores **correlations and dependencies** among features.
- Detects **patterns, clusters, or trends** that may guide feature engineering or hypothesis generation.

Real-World Example

Let's analyze an online retail dataset containing InvoiceNo, CustomerID, Quantity, Price, and Country.

Through EDA:

- Detect that some transactions have **negative quantities** (returns) → handle appropriately.
- Discover that **average price** differs by **country**, indicating regional pricing strategies.
- Observe that a few customers have **very high spending**—potential outliers or loyal VIP customers.
- Realize that some invoices are missing customer IDs → decide whether to drop or impute.

Such insights allow to **refine your dataset**, ensuring that subsequent predictive models (like sales forecasting or customer segmentation) are reliable and interpretable.

Objectives of EDA

- **Structure:** Identify variables, types (numeric, categorical, datetime, text), units, and ranges.
- **Quality:** Detect missing values, outliers, duplicates, inconsistent formats, and measurement errors.
- **Distribution & Shape:** Understand central tendency, spread, skewness, kurtosis.
- **Relationships:** Explore associations, dependencies, multicollinearity, and interactions.
- **Assumptions for Modeling:** Linearity, independence, normality, homoscedasticity.
- **Feature Ideas:** Transformations, binning, encodings, engineered ratios, and aggregates.
- **Readiness:** Decide data cleaning steps and whether additional data is needed.

Typical EDA Workflow

1. **Data Intake:** Load data; check dimensions, schema, memory, sample rows.
2. **Type & Domain Checks:** Ensure each variable has the correct dtype and plausible values.
3. **Univariate EDA:** Distributions and summary statistics per variable.
4. **Bivariate EDA:** Pairs of variables (target vs. predictor, numeric-numeric, numeric-categorical, categorical-categorical).
5. **Multivariate EDA:** Many variables together (correlations, dimensionality reduction, clustering).
6. **Data Quality:** Missingness patterns, outliers, duplicates, inconsistent categories.
7. **Feature & Assumption Checks:** Transformations, encodings, interaction terms; check modeling assumptions.
8. **Documentation:** Record insights, issues, and decisions (a short EDA report with figures/tables).

EDA is more than a preparatory step—it is an **iterative, insight-driven process**. By combining **statistical summaries**, **graphical exploration**, and **data cleaning**, analysts can turn unstructured, messy data into a well-understood foundation for deeper modeling and decision-making.

1.1.1 Types of Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) can be divided into **different types** based on **what we are trying to find out** from the data. Each type helps us explore the dataset from a different angle — like checking overall shape, comparing features, or spotting relationships.

The main **types of EDA** are:

1. **Descriptive Analysis**
2. **Univariate Analysis**
3. **Bivariate Analysis**
4. **Multivariate Analysis**
5. **Graphical and Non-Graphical EDA**

1. Descriptive EDA (Descriptive Analysis)

Descriptive EDA helps us **summarize and describe** the main features of a dataset. It gives a **quick overview** of how the data looks — its size, values, and patterns — without doing any complex modeling. It helps to

- Find the **basic statistics** of each column, like:
 - Minimum, Maximum
 - Mean (average), Median (middle value), Mode (most frequent)
 - Standard deviation (how spread the data is)
- Count how many **rows and columns** the dataset has.
- Check **data types** (numeric, text, date, etc.).
- Find out if there are **missing values** or **duplicate records**.

Tools used:

- In Python: `df.describe()`, `df.info()` (in Pandas)
- In Excel: “Summary Statistics” or “Data Analysis” tool

Example:

A dataset of students with columns like *Age*, *Marks*, and *Gender*, Descriptive EDA will find the following:

- Average age = 20.5 years
- Highest marks = 98
- Male = 60%, Female = 40%

Example:

<pre>import pandas as pd # 1 Create a small dataset of students data = {</pre>	Output:
---	----------------

<pre> "Name": ["Asha", "Rahul", "Priya", "Aman", "Navin"], "Age": [19, 20, 21, 22, 20], "Marks": [78, 85, 98, 67, 90], "Gender": ["Female", "Male", "Female", "Male", "Male"] } df = pd.DataFrame(data) print("=== First few rows of data ===") print(df) # 2 Basic info about the dataset print("\n=== df.info() ===") print(df.info()) # 3 Descriptive statistics for numeric columns print("\n=== df.describe() (numeric columns) ===") print(df.describe()) </pre>	<pre> === First few rows of data === Name Age Marks Gender 0 Asha 19 78 Female 1 Rahul 20 85 Male 2 Priya 21 98 Female 3 Aman 22 67 Male 4 Navin 20 90 Male === df.info() === <class 'pandas.core.frame.DataFrame'> RangeIndex: 5 entries, 0 to 4 Data columns (total 4 columns): # Column Non-Null Count Dtype --- -- 0 Name 5 non-null object 1 Age 5 non-null int64 2 Marks 5 non-null int64 3 Gender 5 non-null object dtypes: int64(2), object(2) memory usage: 288.0+ bytes None === df.describe() (numeric columns) === Age Marks count 5.000000 5.000000 mean 20.400000 83.600000 std 1.140175 11.802542 min 19.000000 67.000000 25% 20.000000 78.000000 50% 20.000000 85.000000 75% 21.000000 90.000000 max 22.000000 98.000000 </pre>
--	--

2. Univariate EDA (Single Variable Analysis)

“Uni” means **one**.

Univariate analysis studies **one variable at a time** to understand its **distribution and nature**.

Main Functionality

- For **numerical data** (like Age, Salary, Marks):
 - Find mean, median, mode, range, variance, standard deviation.
 - Make **histograms**, **box plots**, or **density plots** to see how values are spread.
- For **categorical data** (like Gender, City, Grade):
 - Count frequency (how many times each category appears).
 - Make **bar charts** or **pie charts**.

It is suitable, if the data is:

- **Normal or skewed** (leaning left or right)
- Has **outliers** (very high or very low values)
- **Imbalanced** (some categories appear too often)

Example:

In a dataset of employees:

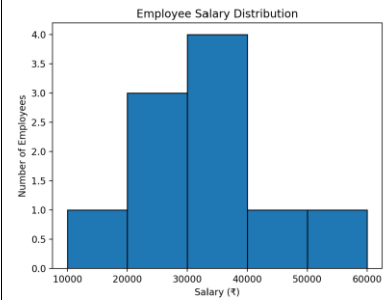
- Salary Variable Analysis shows most salaries between ₹20,000–₹40,000.

Example:

<pre>import pandas as pd import matplotlib.pyplot as plt # 1 Create a small employee salary dataset data = { "Employee": ["E1", "E2", "E3", "E4", "E5", "E6", "E7", "E8", "E9", "E10"], "Salary": [18000, 22000, 25000, 27000, 30000, 32000, 35000, 38000, 42000, 50000] } df = pd.DataFrame(data) print("=== Salary data ===") print(df) # 2 Plot a histogram of salaries plt.hist(df["Salary"], bins=[10000, 20000, 30000, 40000, 50000, 60000], edgecolor="black") plt.title("Employee Salary Distribution")</pre>	Output: <pre>=== Salary data === Employee Salary 0 E1 18000 1 E2 22000 2 E3 25000 3 E4 27000 4 E5 30000 5 E6 32000 6 E7 35000 7 E8 38000 8 E9 42000 9 E10 50000</pre> Minimum salary: 18000 Maximum salary: 50000 Average salary: 31900.0
---	---

```
plt.xlabel("Salary (₹)")
plt.ylabel("Number of Employees")
plt.show()

# 3 Simple descriptive check
print("\nMinimum salary:", df["Salary"].min())
print("Maximum salary:", df["Salary"].max())
print("Average salary:", df["Salary"].mean())
```



3. Bivariate EDA (Two Variable Analysis)

“Bi” means **two**.

Bivariate analysis studies the **relationship between two variables** at a time — for example, *Age vs Salary*, or *Study Hours vs Marks*.

Main Functionality

Depending on variable types:

Type of Variables	What We Use	What It Shows
Both numeric	Scatter plot, Correlation	How strongly they move together
Numeric + Categorical	Box plot, Grouped bar chart	How categories affect numeric values
Both categorical	Cross table, Heatmap, Chi-square test	How one category relates to another

The method we use depends on the **type of variables**. If **both variables are numeric** (like Age and Salary), we usually use a **scatter plot** and **correlation** to see how strongly they move together. If we have **one numeric and one categorical variable** (like Marks and Class Section, or Salary and Gender), we use **box plots** or **grouped bar charts** to see how different categories affect the numeric values. If **both variables are categorical** (like Gender and Purchase Type, or City and Payment Mode), we use **cross tables**, **heatmaps**, or sometimes a **chi-square test** to see how one category is related to the other.

Bivariate analysis is very useful to find simple patterns and relationships, such as:

- Do older people **earn more** salary?
- Do students who **study more hours** get **higher marks**?
- Does **gender** affect which product people buy?

In short, bivariate analysis helps us understand **how one variable changes with another**.

Example: If plot *Hours Studied vs Exam Score*, more hours = higher marks → **positive correlation**.

```
import pandas as pd
```

```
data = {  
    "Hours": [1, 2, 3, 4, 5],  
    "Marks": [40, 50, 60, 70, 85]  
}
```

```
df = pd.DataFrame(data)  
print(df)
```

```
# Correlation (relationship strength)
```

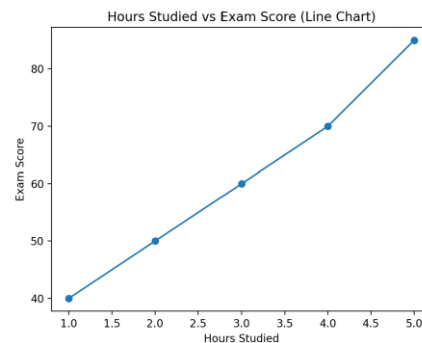
```
print("Correlation:", df["Hours"].corr(df["Marks"]))
```

```
# Line chart
```

```
plt.figure()  
plt.plot(df["Hours"], df["Marks"], marker="o")  
plt.title("Hours Studied vs Exam Score (Line Chart)")  
plt.xlabel("Hours Studied")  
plt.ylabel("Exam Score")  
plt.show()
```

Output:

```
Hours Marks  
0      1    40  
1      2    50  
2      3    60  
3      4    70  
4      5    85  
Correlation: 0.9958932064677037
```



4. Multivariate EDA (Many Variable Analysis)

“Multi” means **many**.

Multivariate analysis studies **more than two variables together** to see how they **interact or influence each other**.

Importance of Multivariate EDA:

Real data usually has **many variables** together, like: Age, Income, Spending, Education, CityType, Occupation, etc.

Multivariate EDA helps to:

- **Find multicollinearity**
→ When two or more variables give almost the **same information** (for example, “Income per month” and “Income per year”).
- **Discover hidden patterns or groups**
→ For example, one group of customers may be “young + high-income + urban”.
- **Identify key features**
→ Which variables **strongly affect** the result (e.g., who will buy more gadgets).

Example:

In a dataset of customers:

- It is found that **young + high-income + urban customers** buy more gadgets. This combined effect is revealed only by multivariate analysis.

```
import pandas as pd
```

```
data = {
    "Age":      [22, 25, 35, 28, 42, 30, 24, 39],
    "Income":    [30000, 45000, 60000, 52000, 75000,
48000, 55000, 70000],
    "CityType":  ["Urban", "Urban", "Rural", "Urban",
    "Rural", "Urban", "Urban", "Rural"],
    "GadgetSpend": [8000, 12000, 5000, 15000, 4000,
13000, 14000, 6000]
}
```

```
df = pd.DataFrame(data)
print("=== Data ===")
print(df)
```

```
# 1) Correlation matrix (numeric variables)
print("\n=== Correlation matrix ===")
print(df[["Age", "Income", "GadgetSpend"]].corr())
```

```
# 2) Multivariate pattern: young + high income + urban
young_high_urban = df[
    (df["Age"] < 30) &
    (df["Income"] > 40000) &
    (df["CityType"] == "Urban")
]
```

```
print("\n=== Young + High Income + Urban customers
===")
```

```
print(young_high_urban[["Age", "Income", "CityType",
"GadgetSpend"]])
```

```
# -----
```

```
# 4) Scatter Plot 1: Age vs Income & Age Vs GadgetSpend
```

```
# -----
```

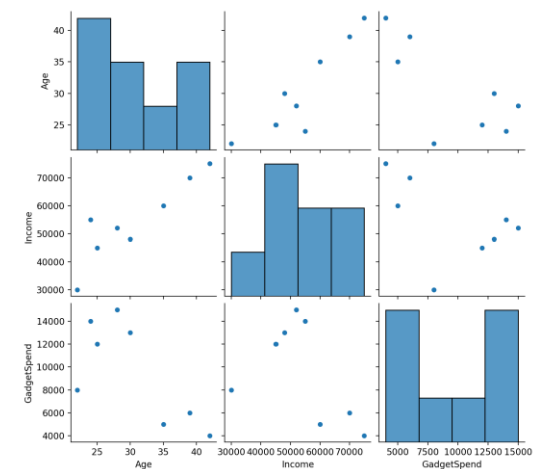
```
plt.figure()
plt.scatter(df["Age"], df["Income"], marker='o')
plt.scatter(df["Age"], df["GadgetSpend"], marker='^')
plt.xlabel("Age")
plt.ylabel("Income & GadgetSpend")
plt.title("Scatter Plot: Age vs Income & Age Vs
GadgetSpend ")
plt.grid(True)
plt.show()
```

Output:

```
=== Data ===
   Age  Income  CityType  GadgetSpend
0    22   30000    Urban         8000
1    25   45000    Urban        12000
2    35   60000    Rural         5000
3    28   52000    Urban        15000
4    42   75000    Rural         4000
5    30   48000    Urban        13000
6    24   55000    Urban        14000
7    39   70000    Rural         6000

=== Correlation matrix ===
           Age      Income  GadgetSpend
Age      1.000000  0.899077 -0.709360
Income   0.899077  1.000000 -0.486548
GadgetSpend -0.709360 -0.486548  1.000000

=== Young + High Income + Urban customers ===
   Age  Income  CityType  GadgetSpend
1    25   45000    Urban        12000
3    28   52000    Urban        15000
6    24   55000    Urban        14000
```



- Creates a small **customer dataset** with Age, Income, CityType and GadgetSpend.
- Prints the **correlation matrix** for Age, Income, and GadgetSpend
→ This tells how these numeric variables are related.
- Then it **filters** the rows where:
 - Age < 30 (young)
 - Income > 40000 (high income)
 - CityType = "Urban"

This shows the **group of young, high-income, urban customers** and their gadget spending.

5. Graphical vs. Non-Graphical EDA

EDA can also be divided based on **how information is displayed**:

A. Graphical EDA

Uses **charts and plots** to visually understand patterns.

- **Histogram**: shows frequency of numeric values.
- **Box plot**: shows spread and outliers.
- **Scatter plot**: shows relationships between two variables.
- **Bar chart / Pie chart**: shows category proportions.
- **Heatmap**: shows correlations in color form.

B. Non-Graphical EDA

Uses **numbers and summaries** instead of pictures.

- Mean, median, mode
- Variance, standard deviation, range
- Frequency tables, correlation coefficients

Note: Non-graphical EDA is useful for statistical summaries and reporting.

Type	Description	Focus	Method / Tools	Example
Descriptive	"What is in my data?"	Overall summary	Mean, median, std, describe()	Average marks of class
Univariate	"How does each variable behave alone?"	One variable	Histogram, boxplot, bar chart	Age distribution
Bivariate	"How do two variables relate?"	Two variables	Scatter plot, correlation	Study time vs Marks
Multivariate	"How do multiple variables interact together?"	More than two variables	Scatter, Pair plot, PCA, Clustering	Age, Salary, Spending together
Graphical	"How can I best see and summarize this	Visual methods	Histogram, pie chart, heatmap	Trend visualization

Chapter 1 Fundamentals of Data Analytics

	data?"			
Non-Graphical	"What is in my data?"	Statistical summaries	Mean, median, mode, correlation table	Summary statistics

Example: For student dataset:

Name	Gender	Age	Hours_Studied	Marks
Asha	F	20	4	75
Ravi	M	22	3	65
Meena	F	21	5	85
Karan	M	19	2	55

Type	Observations
Descriptive	Average marks = 70, Average hours = 3.5
Univariate	Age ranges from 19–22; Marks mostly between 55–85
Bivariate	Higher study hours → higher marks
Multivariate	Girls study more hours and score higher marks
Graphical	Scatter plot shows upward trend; bar chart shows gender distribution

1.1.5 Handling Missing Data and Outliers

Missing data means some entries (values) in a dataset are **empty, blank, or undefined**. These appear as NaN (Not a Number) or None in Python.

Example: Here, Age, Salary, and City have missing values.

Name	Age	Salary	City
Riya	25	40000	Surat
Rahul	NaN	35000	NaN
Meena	30	NaN	Ahmedabad

Detecting Missing Data in Python

Example with a simple dataset:

<pre>import pandas as pd import numpy as np # Sample data data = { 'Name': ['Riya', 'Rahul', 'Meena', 'Karan', 'Tina'], 'Age': [25, np.nan, 30, 27, np.nan], 'Salary': [40000, 35000, np.nan, 50000, 45000], 'City': ['Surat', np.nan, 'Ahmedabad', 'Vadodara', 'Surat'] } df = pd.DataFrame(data) print(df)</pre>	Output <pre> Name Age Salary City 0 Riya 25.0 40000.0 Surat 1 Rahul NaN 35000.0 NaN 2 Meena 30.0 NaN Ahmedabad 3 Karan 27.0 50000.0 Vadodara 4 Tina NaN 45000.0 Surat</pre>
---	--

Checking Missing Values

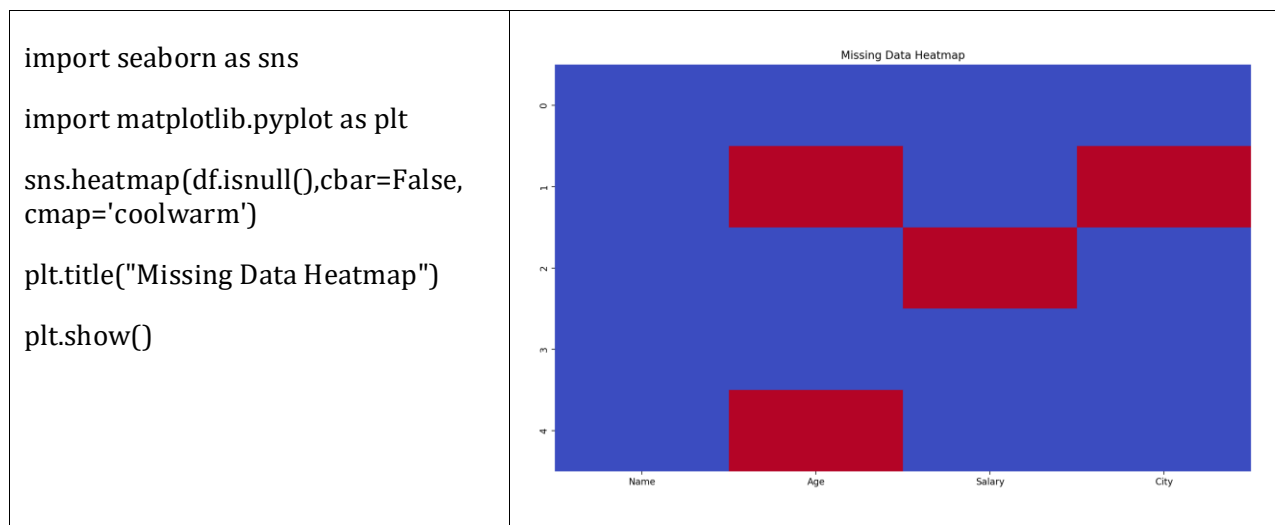
To check the missing values below functions are used:

<pre># Check if any value is missing df.isnull() # Count missing values column-wise df.isnull().sum() # Percentage of missing values (df.isnull().sum() / len(df)) * 100</pre>	Output: <pre> Name Age Salary City 0 False False False False 1 False True False True 2 False False True False 3 False False False False 4 False True False False Name 0 Age 2 Salary 1 City 1 dtype: int64 Name 0.0 Age 40.0 Salary 20.0 City 20.0 dtype: float64</pre>
---	--

This shows **how much data is missing** in each column.

Visualizing Missing Data

Seaborn library is use to find the missing patterns visually.



Interpretation

The colored boxes show which values are missing in red color.

Ways to Handle Missing Data

INITIAL ARRANGEMENTS: Import Libraries and Create Data Frame

<pre>import pandas as pd import numpy as np # Sample dataset data = { 'Name': ['Riya', 'Rahul', 'Meena', 'Karan', 'Tina'], 'Age': [25, np.nan, 30, np.nan, 28], 'Salary': [40000, 35000, np.nan, 50000, np.nan], 'City': ['Surat', np.nan, 'Ahmedabad', 'Vadodara', np.nan] } df = pd.DataFrame(data) print("Original DataFrame:\n", df)</pre>	Output : Original DataFrame: <table><tr><th></th><th>Name</th><th>Age</th><th>Salary</th><th>City</th></tr><tr><td>0</td><td>Riya</td><td>25.0</td><td>40000.0</td><td>Surat</td></tr><tr><td>1</td><td>Rahul</td><td>NaN</td><td>35000.0</td><td>NaN</td></tr><tr><td>2</td><td>Meena</td><td>30.0</td><td>NaN</td><td>Ahmedabad</td></tr><tr><td>3</td><td>Karan</td><td>NaN</td><td>50000.0</td><td>Vadodara</td></tr><tr><td>4</td><td>Tina</td><td>28.0</td><td>NaN</td><td>NaN</td></tr></table>		Name	Age	Salary	City	0	Riya	25.0	40000.0	Surat	1	Rahul	NaN	35000.0	NaN	2	Meena	30.0	NaN	Ahmedabad	3	Karan	NaN	50000.0	Vadodara	4	Tina	28.0	NaN	NaN
	Name	Age	Salary	City																											
0	Riya	25.0	40000.0	Surat																											
1	Rahul	NaN	35000.0	NaN																											
2	Meena	30.0	NaN	Ahmedabad																											
3	Karan	NaN	50000.0	Vadodara																											
4	Tina	28.0	NaN	NaN																											

A. Deletion Methods

Deletion means **removing rows or columns** that contain missing (NaN) values.

1 Remove Rows with Missing Values

Concept:

If only a few rows in the dataset have missing values, we can delete them. This method is called **listwise deletion**.

When to Use:

- When missing data is very small (less than 5% of total records).
- When every feature in that row is incomplete.
- When deletion will not affect data balance.

Example:

<pre>df_dropped_rows = df.dropna() print("After Dropping Rows:\n", df_dropped_rows)</pre>	<pre>After Dropping Rows: Name Age Salary City 0 Riya 25.0 40000.0 Surat</pre>
--	--

Interpretation:

Only the first row is complete (no NaN), so all other rows were deleted.

2. Remove Columns with Too Many Missing Values

Concept: If a column has too many missing entries, it may not be useful. Then the entire column can be dropped.

When to Use:

- When more than 70–80% values in a column are missing.
- When the feature does not provide much information.

Example: We will remove columns that have less than 3 non-null values (i.e., more than 40% missing).

```
df_dropped_cols = df.dropna(axis=1, thresh=3)
print("After Dropping Columns:\n", df_dropped_cols)
```

Output:

```
After Dropping Columns:
   Name  Age  Salary
0  Riya  25.0  40000.0
1  Rahul  NaN  35000.0
2  Meena  30.0     NaN
3  Karan  NaN  50000.0
4   Tina  28.0     NaN
```

Interpretation:

The 'City' column was dropped because it had more than 2 missing values.

B. Imputation (Filling) Methods

Instead of removing data, we can **fill (impute)** missing values using **logical or statistical techniques**. This preserves all rows and keeps the dataset size unchanged.

Example:

```
df['Age'].fillna(df['Age'].mean(), inplace=True)
print("After Mean Imputation on Age:\n", df)
```

```
After Mean Imputation on Age:
   Name  Age  Salary  City
0  Riya  25.000000  40000.0  NaN
1  Rahul  27.666667  35000.0  NaN
2  Meena  30.000000     NaN  Ahmedabad
3  Karan  27.666667  50000.0  Vadodara
4   Tina  28.000000     NaN     NaN
```

Interpretation:

The missing Age values were replaced with the **mean age = 27.67**.

(i) Fill with Mean (Numerical Data)

Replaces missing numeric values with the **average (mean)** of that column. Useful when data distribution is **symmetric (normal)**.

Example:

```
df['Age'].fillna(df['Age'].mean(), inplace=True)
print("After Mean Imputation on Age:\n", df)
```

```
After Mean Imputation on Age:
   Name  Age  Salary  City
0  Riya  25.000000  40000.0  NaN
1  Rahul  27.666667  35000.0  NaN
2  Meena  30.000000   NaN  Ahmedabad
3  Karan  27.666667  50000.0  Vadodara
4   Tina  28.000000   NaN   NaN
```

Interpretation:

The **median salary = 40000**, used to replace missing salary values.

(ii) Fill with Median (Numerical Data)

Replaces missing values with the **middle value (median)**. Best when data is **skewed or contains outliers**.

```
df['Salary'].fillna(df['Salary'].median(),
inplace=True)
print("After Median Imputation on Salary:\n", df)
```

```
After Median Imputation on Salary:
   Name  Age  Salary  City
0  Riya  25.000000  40000.0  NaN
1  Rahul  27.666667  35000.0  NaN
2  Meena  30.000000  40000.0  Ahmedabad
3  Karan  27.666667  50000.0  Vadodara
4   Tina  28.000000  40000.0  NaN
```

Interpretation:

The **median salary = 40000**, used to replace missing salary values.

(iii) Fill with Mode (Categorical Data)

Replaces missing values with the **most frequent (mode)** category.

Example: If most people are from “Surat,” missing city values can be filled with “Surat.”

```
df['City'].fillna(df['City'].mode()[0], inplace=True)
print("After Mode Imputation on City:\n", df)
```

```
After Mode Imputation on City:
   Name  Age  Salary  City
0  Riya  25.000000  40000.0  Ahmedabad
1  Rahul  27.666667  35000.0  Ahmedabad
2  Meena  30.000000  40000.0  Ahmedabad
3  Karan  27.666667  50000.0  Vadodara
4   Tina  28.000000  40000.0  Ahmedabad
```

Interpretation:

Most frequent city is ‘**Surat**’, so all missing city values are replaced with it.

(iv) Forward Fill (ffill)

Copies the **previous (last known)** value into the next missing position. Commonly used for **time-series** data.

Example:

<pre>print("Forward Fill replace missing values with previous ones") df['Age'] = df['Age'].ffill() print(df)</pre>	<pre>Original DataFrame: Name Age Salary City 0 Riya 25.0 40000.0 NaN 1 Rahul NaN 35000.0 NaN 2 Meena 30.0 NaN Ahmedabad 3 Karan NaN 50000.0 Vadodara 4 Tina 28.0 NaN NaN Forward Fill replace missing values with previous ones Name Age Salary City 0 Riya 25.0 40000.0 NaN 1 Rahul 25.0 35000.0 NaN 2 Meena 30.0 NaN Ahmedabad 3 Karan 30.0 50000.0 Vadodara 4 Tina 28.0 NaN NaN</pre>
--	--

If there were missing values, they would be replaced by the **previous non-missing** value.

(v) Backward Fill (bfill)

Copies the **next available** value to fill previous missing data.

<pre>print("Backward Fill replace missing values with next ones") df['Salary'] = df['Salary'].bfill() print(df)</pre>	<pre>Backward Fill replace missing values with next ones Name Age Salary City 0 Riya 25.0 40000.0 NaN 1 Rahul 25.0 35000.0 NaN 2 Meena 30.0 50000.0 Ahmedabad 3 Karan 30.0 50000.0 Vadodara 4 Tina 28.0 NaN NaN</pre>
---	--

If there were missing values at the top, they would be replaced with the **next available** value.

C. Advanced Imputation

Simple filling methods like **mean**, **median**, or **mode** are not enough. In real life, data is more complicated, and different columns may **depend on each other**. Advanced Imputation uses **smart algorithms** to guess the missing values more accurately.

For Example: If **Age**, **Salary**, and **HoursStudied** all have missing values, simple filling may not give the best results.

There are **three important advanced methods**:

1. KNN Imputer
2. Iterative Imputer
3. Regression Imputer

1) KNN Imputer (K-Nearest Neighbors Imputation)

- This method looks at **other rows that are similar** (nearest neighbors).
- It checks which rows have values close to your missing row.
- Then it uses their average to fill the missing values

It is a better method as compared to other methods, because it does not blindly use mean/median. It uses **similar people/rows** to decide the value.

Example:

```
import pandas as pd
import numpy as np
from sklearn.impute import KNNImputer

# -----
# Create a very easy DataFrame
# -----
data = {
    "Name": ["Riya", "Rahul", "Meena", "Karan", "Tina"],
    "Age": [25, 28, 30, 26, 27],
    "HoursStudied": [4.0, 3.5, 5.0, 2.0, 4.5],
    "Temperature": [30.0, 31.2, 29.8, 32.5, 30.5],
    "Salary": [40000, 35000, 45000, 50000, np.nan]
}
# Tina's salary missing
df = pd.DataFrame(data)

print("Original DataFrame (Tina's Salary Missing) \n", df)

# -----
# Apply KNN Imputer
# -----
num_cols = ["Age", "HoursStudied", "Temperature", "Salary"]

imputer = KNNImputer(n_neighbors=3)
df_imputed = df.copy()
df_imputed[num_cols] = imputer.fit_transform(df_imputed[num_cols])

print("After KNN Imputation (Tina's Salary Filled) \n", df_imputed)
```

Original DataFrame (Tina's Salary Missing)					
	Name	Age	HoursStudied	Temperature	Salary
0	Riya	25	4.0	30.0	40000.0
1	Rahul	28	3.5	31.2	35000.0
2	Meena	30	5.0	29.8	45000.0
3	Karan	26	2.0	32.5	50000.0
4	Tina	27	4.5	30.5	NaN

After KNN Imputation (Tina's Salary Filled)					
	Name	Age	HoursStudied	Temperature	Salary
0	Riya	25.0	4.0	30.0	40000.0
1	Rahul	28.0	3.5	31.2	35000.0
2	Meena	30.0	5.0	29.8	45000.0
3	Karan	26.0	2.0	32.5	50000.0
4	Tina	27.0	4.5	30.5	40000.0

Interpretation:

Tina's Salary was missing → KNN looked at the 3 most similar people based on:

- Age
- HoursStudied
- Temperature

Then it used their salary values to **estimate Tina's Salary**. KNN imputer filled Tina's Salary as **₹40,000**.

2) MICE / Iterative Imputer

(MICE = Multiple Imputation by Chained Equations)

- This method fills missing values **one column at a time**, using **all other columns** to help.
- It repeats the filling process many times (iterations) until the values stabilize.

As it uses all available information, Iterative Imputer works well when many columns have missing values.

Example: If *Meena's Age* is missing → MICE looks at **Salary, HoursStudied, Temperature** and predicts a good value. It does this again and again (iteratively) until the values become stable.

Interpretation: How MICE Filled the Missing Values?

Meena's Age was missing. MICE looked at: HoursStudied = 5, Temperature = 29.8, Salary = 45000
Using these values, it predicted a realistic Age close to 22-25.

MICE is smarter than mean/median because:

- It uses **all relationships** between columns
- It repeats predictions until everything stabilizes

- It gives more **realistic** values

```
import pandas as pd
import numpy as np
from sklearn.impute import KNNImputer
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer

# -----
# Create a very easy DataFrame
# -----
data = {
    "Name": ["Riya", "Rahul", "Meena", "Karan", "Tina"],
    "Age": [25, 28, np.nan, 26, 27],
    "HoursStudied": [4.0, 3.5, 5.0, 2.0, 4.5],
    "Temperature": [30.0, 31.2, 29.8, 32.5, 30.5],
    "Salary": [40000, 35000, 45000, 50000, 35000]
}
df = pd.DataFrame(data)
print(df)

#only numeric columns
num_cols = ["Age", "HoursStudied", "Temperature", "Salary"]

# Step 2: Create Imputer
mice = IterativeImputer(random_state=0)

# Step 3: Fit and transform
df_mice = df.copy()
df_mice[num_cols] = mice.fit_transform(df_mice[num_cols])

print("\nAfter MICE Imputation:\n", df_mice)
```

	Name	Age	HoursStudied	Temperature	Salary
0	Riya	25.0	4.0	30.0	40000
1	Rahul	28.0	3.5	31.2	35000
2	Meena	NaN	5.0	29.8	45000
3	Karan	26.0	2.0	32.5	50000
4	Tina	27.0	4.5	30.5	35000

After MICE Imputation:

	Name	Age	HoursStudied	Temperature	Salary
0	Riya	25.000000	4.0	30.0	40000.0
1	Rahul	28.000000	3.5	31.2	35000.0
2	Meena	23.428561	5.0	29.8	45000.0
3	Karan	26.000000	2.0	32.5	50000.0
4	Tina	27.000000	4.5	30.5	35000.0

3) Regression Imputation

Regression Imputation is a technique used to fill missing values by **predicting** them using a statistical or machine learning **regression model**. Instead of using simple averages, this method learns a **relationship between variables**.

Example: Tina's Salary is missing.

```
df_reg = df.copy()

# Independent variables (features)
X = df_reg[["Age", "HoursStudied", "Temperature"]]

# Dependent variable (target)
y = df_reg["Salary"]

# Rows where salary is not missing (training data)
mask_train = y.notna()

model = LinearRegression()
model.fit(X[mask_train], y[mask_train])

# Rows where salary is missing (prediction)
mask_missing = y.isna()

# Predict Tina's salary
predicted_salary = model.predict(X[mask_missing])

df_reg.loc[mask_missing, "Salary"] = predicted_salary

print("\nAfter Regression-Based Imputation:\n", df_reg)
```

Output

	Name	Age	HoursStudied	Temperature	Salary
0	Riya	25	4.0	30.0	40000.0
1	Rahul	28	3.5	31.2	35000.0
2	Meena	23	5.0	29.8	45000.0
3	Karan	26	2.0	32.5	50000.0
4	Tina	27	4.5	30.5	NaN

After Regression-Based Imputation:

	Name	Age	HoursStudied	Temperature	Salary
0	Riya	25	4.0	30.0	40000.0
1	Rahul	28	3.5	31.2	35000.0
2	Meena	23	5.0	29.8	45000.0
3	Karan	26	2.0	32.5	50000.0
4	Tina	27	4.5	30.5	34200.0

Interpretation: A **Linear Regression model is trained** using all other people:

- Age
- HoursStudied
- Temperature
- Salary

The model learned patterns like:

- Higher HoursStudied → higher Salary
- Higher Temperature (indicating tougher conditions) → maybe higher Salary
- Age sometimes relates to experience

Using this learning, the model **predicted Tina's Salary**.

1.5 What Are Outliers?

Outliers are values in a dataset that are extremely high or extremely low compared to all other values. Outliers *do not fit* with the normal pattern of the data.

For Example:

If most students score between **40 to 80 marks**, but one student scores **2 marks** or **100 marks**,
→ These unusual values are **outliers**. If monthly salaries are between **₹30,000 and ₹60,000**,
but one employee earns **₹5,00,000**,
→ This is an **outlier**.

Outliers happen due to:

- Mistakes in data entry
- Special events
- Unusual behavior
- Machine or sensor errors
- Natural extreme values

Outliers are important because they can:

- Pull the **average** too high or low
- Distort **charts and graphs**
- Mislead **machine learning models**
- Change **results and conclusions**

So, Outliers should be detected and handled properly. Thus, Outliers are extreme values that lie far away from the rest of the data and do not follow the usual pattern.

1. Remove Outliers (Delete the row)

If the programmer is sure that the outlier is a **mistake**, simply delete that record.

Example:

- Age = **200** (impossible)
- Salary = **-1000** (negative)

These are typing errors → remove them.

```
import pandas as pd

# Creating a simple DataFrame with outliers
data = {
    "Name": ["Riya", "Rahul", "Meena", "Karan"],
    "Age": [25, 30, 200, 27],
    "Salary": [40000, -1000, 45000, 50000]
}

# 200 is an outlier (impossible age)
# -1000 is an outlier (negative salary)

df = pd.DataFrame(data)

print("Original DataFrame:")
print(df)

# --- Removing Outliers ---
# Condition: Keep only rows where Age < 120 and Salary > 0

df_clean = df[(df["Age"] < 120) & (df["Salary"] > 0)]

print("\nDataFrame After Removing Outliers:")

print(df_clean)
```

Output :

```
Original DataFrame:
   Name  Age  Salary
0  Riya   25   40000
1  Rahul  30   -1000
2  Meena 200   45000
3  Karan  27   50000

DataFrame After Removing Outliers:
   Name  Age  Salary
0  Riya   25   40000
3  Karan  27   50000
```

It is used when:

- ✓ Outliers are few
- ✓ Outliers are clearly wrong
- ✓ Data size is large

2. Replace Outliers (Capping)

Instead of deleting, replace extreme values with a **safe limit**.

Example: If normal salary range is ₹20,000 to ₹80,000:

- If someone has ₹5,00,000 → replace with ₹80,000
- If salary is ₹5,000 → replace with ₹20,000

This method keeps all rows.

<pre># Sample DataFrame with outliers data = { "Name": ["Riya", "Rahul", "Meena", "Karan", "Tina"], "Salary": [30000, 5000, 45000, 500000, 25000] # 5000 and 500000 are outliers } df = pd.DataFrame(data) print("Original DataFrame:") print(df) # --- Capping (Replace Outliers with Limits) --- min_limit = 20000 max_limit = 80000 df["Salary"] = df["Salary"].clip(lower=min_limit, upper=max_limit) print("\nAfter Capping Outliers:") print(df)</pre>	Output: Original DataFrame: <table><tr><th></th><th>Name</th><th>Salary</th></tr><tr><td>0</td><td>Riya</td><td>30000</td></tr><tr><td>1</td><td>Rahul</td><td>5000</td></tr><tr><td>2</td><td>Meena</td><td>45000</td></tr><tr><td>3</td><td>Karan</td><td>500000</td></tr><tr><td>4</td><td>Tina</td><td>25000</td></tr></table> After Capping Outliers: <table><tr><th></th><th>Name</th><th>Salary</th></tr><tr><td>0</td><td>Riya</td><td>30000</td></tr><tr><td>1</td><td>Rahul</td><td>20000</td></tr><tr><td>2</td><td>Meena</td><td>45000</td></tr><tr><td>3</td><td>Karan</td><td>80000</td></tr><tr><td>4</td><td>Tina</td><td>25000</td></tr></table>		Name	Salary	0	Riya	30000	1	Rahul	5000	2	Meena	45000	3	Karan	500000	4	Tina	25000		Name	Salary	0	Riya	30000	1	Rahul	20000	2	Meena	45000	3	Karan	80000	4	Tina	25000
	Name	Salary																																			
0	Riya	30000																																			
1	Rahul	5000																																			
2	Meena	45000																																			
3	Karan	500000																																			
4	Tina	25000																																			
	Name	Salary																																			
0	Riya	30000																																			
1	Rahul	20000																																			
2	Meena	45000																																			
3	Karan	80000																																			
4	Tina	25000																																			

When to use:

- ✓ When you want to keep data
- ✓ When extreme values distort results

3. Transform the Data (Log, Square Root)

Sometimes numbers vary too much. Math Functions can be applied to make big numbers smaller:

Example:

- `log(salary)`
- `sqrt(salary)`

This reduces the effect of very large values.

<pre>import pandas as pd import numpy as np</pre>	Output:
---	-----------------------

<pre>data = { "Name": ["Riya", "Rahul", "Meena", "Karan"], "Salary": [30000, 40000, 600000, 900000] # large salaries = outliers } df = pd.DataFrame(data) print("Original DataFrame:") print(df) df["Salary_sqrt"] = np.sqrt(df["Salary"]) df["Salary_sqrt"] = df["Salary_sqrt"].round(2) print("\nAfter Square Root Transformation:") print(df[["Name", "Salary", "Salary_sqrt"]])</pre>	<pre>Original DataFrame: Name Salary 0 Riya 30000 1 Rahul 40000 2 Meena 600000 3 Karan 900000 After Square Root Transformation: Name Salary Salary_sqrt 0 Riya 30000 173.21 1 Rahul 40000 200.00 2 Meena 600000 774.60 3 Karan 900000 948.68</pre>
--	--

4. Use Median Instead of Mean

Mean (average) gets easily affected by outliers. Median (middle value) is **not affected**.

Example:

Marks: 20, 25, 30, 35, 32, 99

- Mean becomes large
- Median remains stable

<pre>marks = [20, 25, 30, 35, 32, 99] mean_value = sum(marks) / len(marks) print("Mean:", mean_value)</pre>	Output: Mean: 40.1666...
---	--

The average becomes **40** because the outlier **99** pulls the mean upward.

5. Analyze Outliers Separately

Outliers are not always mistakes. Sometimes they show **important information** that must be studied carefully. So instead of removing them, **analyze them separately**.

Example:

- High sales during Diwali
- Abnormal bank transactions (fraud detection)
- Sudden spike in website traffic

<pre>import pandas as pd</pre>	Output:
--------------------------------	----------------

```
data = {"Visitors": [100, 120, 110, 115, 800]}
# 800 = outlier

df = pd.DataFrame(data)
# Identify outliers (simple rule: > 2x average)

outliers = df[df["Visitors"] > 2 * df["Visitors"].mean()]
normal = df[df["Visitors"] <= 2 * df["Visitors"].mean()]

print("Normal Data:")
print(normal)

print("\nOutliers to Analyze Separately:")
print(outliers)
```

```
Normal Data:
  Visitors
0       100
1       120
2       110
3       115

Outliers to Analyze Separately:
  Visitors
4       800
```

In such cases:

- ✓ Do NOT remove outliers
- ✓ Study them separately because they give useful insights

1.2 Understanding the Data

1.2.1 Quantitative Data: Discrete and Continuous

1. Quantitative data refers to **numerical** data that can be **measured or counted**.
2. It answers the questions “**How much?**” or “**How many?**” and consists of values that can be analyzed using mathematical tools and algorithms.
3. This type of data is often **structured**, making it suitable for tasks such as **trend forecasting, statistical analysis, budgeting, pattern identification, and performance measurement**.

Types of Quantitative Data:

1. Discrete Data:

- **Definition:** Data that can take **only specific, distinct values** (usually whole numbers).
- **Nature:** *Countable* — there are no intermediate or fractional values between two data points.
- **Examples:**
 - Number of students in a class (e.g., 25, 30)
 - Number of cars in a parking lot
 - Number of products sold
 - Test scores (if given as whole numbers)

2. Continuous Data:

- **Definition:** Data that can take **any value within a given range**.
- **Nature:** *Measurable* — can include fractions or decimals.
- **Examples:**
 - Height of a person (e.g., 5.6 feet)

- Temperature (e.g., 37.2°C)
- Time (e.g., 2.45 hours)

Key Differences:

Feature	Discrete Data	Continuous Data
Type of values	Specific, fixed values	Any value in a range
Nature	Countable	Measurable
Examples	Number of books, Test Scores	Weight, distance, time, temperature
Can be fractional?	No	Yes

1.2.2 Qualitative Data: Non-Numerical (Nominal and Ordinal)

Qualitative data refers to **non-numerical information** that describes categories, labels, characteristics, or experiences rather than quantities.

It answers the questions “**What kind?**” or “**Which category?**”, capturing aspects that **numbers cannot measure directly**.

Examples include **customer feedback, product reviews, interview transcripts, and social media comments**.

Forms of Qualitative Data

- **Structured:** Organized and coded (e.g., survey responses with predefined options).
- **Unstructured:** Free-text, descriptive, or narrative (e.g., interview transcripts, open-ended feedback).

Types of Qualitative Data:

1. Nominal Data:

- **Definition:** Data that represents Categories without any inherent order, hierarchy or ranking.
- **Nature:** Labels only — can be counted, but not logically ordered.
- **Examples:**
 - Gender (male, female)
 - Blood group (A, B, AB, O)
 - Eye color (blue, green, brown)
 - Nationality (Indian, American, etc.)
- **Key Point:** We can **name and count** them, but **not rank** them.

2. Ordinal Data:

- **Definition:** Data that represents categories with a meaningful order or ranking, but the **differences between values are not measurable or uniform**.
- Ranking exists, but intervals are not equal or known.
- **Nature:** Order matters, but spacing between levels is inconsistent.
- **Examples:**
 - Education level (High School < Bachelor's < Master's < Ph.D.)

- Customer satisfaction (Very dissatisfied < Neutral < Very satisfied)
- Class rank (1st, 2nd, 3rd)
- Survey rating (1 = Poor, 5 = Excellent)
- **Key Point: Ranking exists, but intervals are not equal.**

Key Differences:

Feature	Nominal Data	Ordinal Data
Order/Ranking	No	Yes
Type of values	Categories only	Ordered categories
Examples	Gender, Blood group, Colors	Satisfaction level, Education Ranks, Survey ratings
Measurement	Labeling & Counting	Ranking without measurable intervals

Common Use Cases of Qualitative Data

- Understanding **customer behavior**
- Analyzing **market trends**
- Studying **user experiences**
- Organizing **feedback and opinions** for improvement

Summary

Data Type	Sub-type	Characteristics	Examples
Quantitative	Discrete	Countable, whole numbers	Number of pets or students
	Continuous	Measurable, includes decimals	Height, weight, temperature
Qualitative	Nominal	Categorical, no order	Gender, nationality, colors
	Ordinal	Categorical with order, no fixed scale	Satisfaction levels, class ranks

1.3 Spread of Data

The spread of data (also known as **variability** or **dispersion**) refers to how much the data values differ or vary from the central value — such as the **mean**, **median**, or **mode**. Understanding the spread helps interpret the **shape**, **distribution**, and **behavior** of a dataset, making it easier to **detect patterns**, **outliers**, or **irregularities**.

1.3.1 Normal Distribution

A **normal distribution**, also known as the **Gaussian distribution** or **bell curve**, is a **symmetrical**, **bell-shaped curve** where most data values **cluster around the mean**, and the **probabilities taper off equally on both sides**.

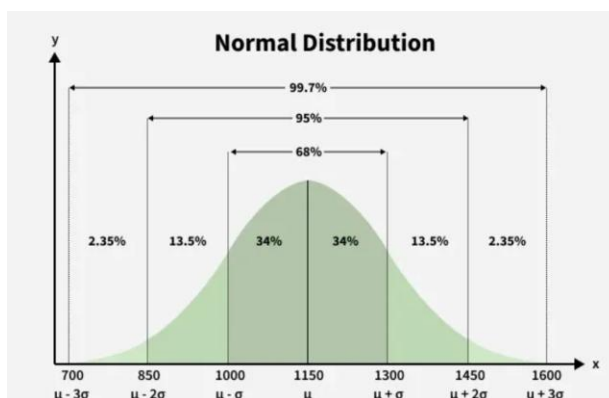
Key Characteristics:

- **Mean = Median = Mode**
- Symmetrical shape: **Symmetric** about the mean (both sides mirror each other)
- **Fully described by two parameters:**
 - Mean (μ): Center of the curve
 - Standard Deviation (σ): Spread or dispersion of the curve

- **Empirical Rule (68-95-99.7 Rule):**
 - ~68% of data lies within $\pm 1\sigma$
 - ~95% within $\pm 2\sigma$
 - ~99.7% within $\pm 3\sigma$
- **Area under the curve = 1** (represents total probability)
- Commonly used in natural and social phenomena — **heights, IQ scores, test results**

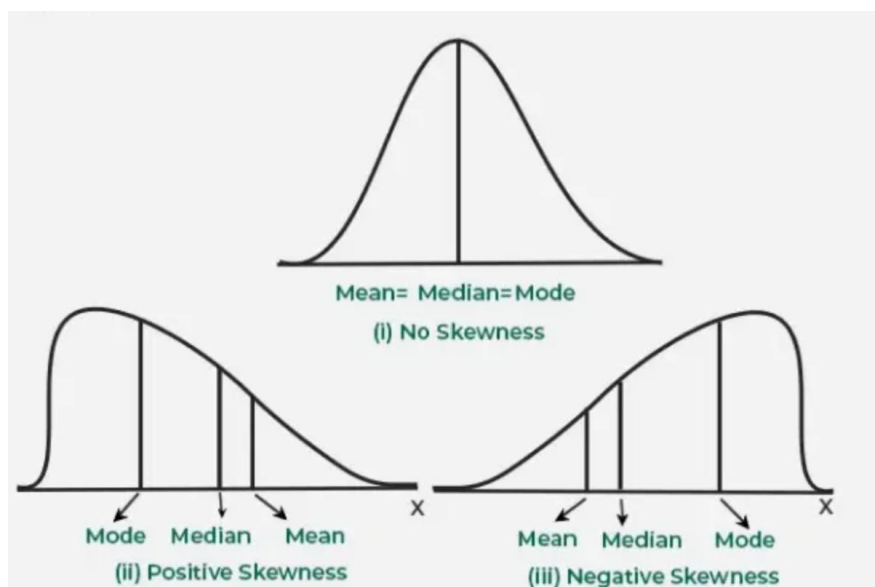
Visualization:

A smooth, **bell-shaped curve** centered on the mean (μ), with tails decreasing symmetrically on both sides.



1.3.2 Skewed Distribution

A skewed distribution is one in which data is not symmetrical — it has a longer tail on one side of the mean.



Types of Skewness

Type	Description	Shape / Example
No Skewness (Symmetrical)	Mean = Median = Mode. Data is balanced on both sides.	Bell-shaped curve (Normal Distribution)
Right-Skewed (Positive Skew)	Long tail on the right side. Most values lie on the left . Mean > Median > Mode.	Income, house prices, waiting times
Left-Skewed (Negative Skew)	Long tail on the left side. Most values lie on the right . Mean < Median < Mode.	Retirement age, easy exam scores

Impact on Central Tendency

- **Right Skew:** Mean > Median
- **Left Skew:** Mean < Median

Visualization:

- Right Skew: Curve peaks on the left, tail stretches right
- Left Skew: Curve peaks on the right, tail stretches left

Importance

- Many statistical tests (e.g., regression, t-tests) assume **normality**.
- Highly skewed data can **distort averages and variance**.
- **Transformation** (log, square root, etc.) can reduce skewness for accurate modeling.

Example of Types of Skewness

```
import matplotlib.pyplot as plt

# 1) No Skewness - peak in the middle (symmetrical)
x_sym = [1, 2, 3, 4, 5]
y_sym = [0, 2, 4, 2, 0]

# 2) Right-Skewed - peak on the left, tail on the right
x_right = [1, 2, 3, 4, 5]
y_right = [4, 3, 2, 1, 0]

# 3) Left-Skewed - peak on the right, tail on the left
x_left = [1, 2, 3, 4, 5]
y_left = [0, 1, 2, 3, 4]

fig, axes = plt.subplots(1, 3, figsize=(10, 3))

# No Skewness
axes[0].plot(x_sym, y_sym, marker="o")
axes[0].set_title("No Skewness\n(Symmetrical)")
axes[0].set_xlabel("Value")
```

```
axes[0].set_ylabel("Frequency")

# Right-Skewed
axes[1].plot(x_right, y_right, marker="o")
axes[1].set_title("Right-Skewed\n(Positive)")
axes[1].set_xlabel("Value")

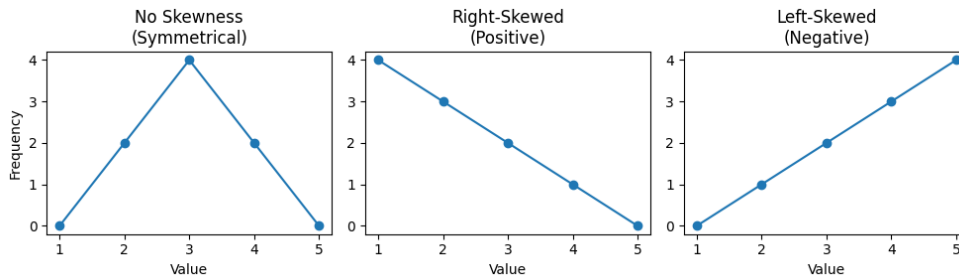
# Left-Skewed
axes[2].plot(x_left, y_left, marker="o")
axes[2].set_title("Left-Skewed\n(Negative)")
axes[2].set_xlabel("Value")
plt.tight_layout()
plt.show()
```

Output:

```
No Skewness (Symmetrical)
Data : [0, 40, 50, 55, 60, 65, 70, 80]
Mean : 52.50
Median: 57.50

Right-Skewed (Positive)
Data : [0, 10, 15, 20, 22, 25, 30, 90]
Mean : 26.50
Median: 21.00

Left-Skewed (Negative)
Data : [0, 70, 75, 80, 85, 90, 95]
Mean : 70.71
Median: 80.00
```



1.3.3 Skewness and Kurtosis

A. Skewness

- **Definition:** Skewness measures the **asymmetry** of the distribution around its mean. It provides value whether data values are more concentrated on one side of the mean.

Skewness Value	Interpretation	Shape
0	Perfectly symmetrical (Normal)	Balanced bell curve
> 0	Right-skewed (Positive Skew)	Tail to the right
< 0	Left-skewed (Negative Skew)	Tail to the left

- **Formula (conceptually)**

$$Skewness = \frac{\sum (x_i - \bar{x})^3}{n \times s^3}$$

Where $\bar{x}$ = mean, s = standard deviation.

B. Kurtosis

- **Definition: Kurtosis** measures the “tailedness” or **peakedness** of a distribution — how heavy or light its tails are compared to the normal distribution.

Types of Kurtosis

Type	Description	Kurtosis Value (Excess Kurtosis)	Interpretation
Mesokurtic	Normal tails (reference distribution)	≈ 0	Platykurtic → Flatter peak, lighter tails (data spread out evenly).
Leptokurtic	Heavy tails, sharp peak (more outliers)	> 0	Mesokurtic → Similar to normal distribution (baseline).
Platykurtic	Light tails, flat peak (fewer outliers)	< 0	Leptokurtic → Sharp central peak, heavy tails (higher chance of outliers).

- **High kurtosis** → More extreme values (outliers), heavier tails, sharper peak.
- **Low kurtosis** → Fewer extreme values, lighter tails, flatter peak.
- Indicates **risk of outliers** and how concentrated data is around the mean.

Formula (Excess Kurtosis)

$$Kurtosis_{excess} = Kurtosis - 3$$

(So that a normal distribution has an excess kurtosis of 0.)

Summary

Concept	Description	Shape / Example
Normal Distribution	Symmetrical bell-shaped curve; mean = median = mode	Heights, IQ scores
Right Skewed Distribution	Tail to the right; mean > median	Income, waiting times
Left Skewed Distribution	Tail to the left; mean < median	Retirement age, easy exams
Skewness	Measures asymmetry of distribution	0 = symmetric; >0 = right skew; <0 = left skew
Kurtosis	Measures tail weight and peak sharpness	>0 = sharp peak; <0 = flat peak

Example for Skewness and Kurtosis

```
import matplotlib.pyplot as plt
import statistics as stats

# 1) Two simple datasets for kurtosis idea
# sharp_peak = many values close to 10
# flat_peak = values spread out

sharp_peak = [10, 10, 10, 9, 11, 10, 10, 10, 8, 12]
flat_peak = [5, 8, 11, 14, 17, 20, 23, 26, 29, 32]

print("Sharp peak data:", sharp_peak)
print("Mean (sharp):", round(stats.mean(sharp_peak), 2))

print("\nFlat peak data:", flat_peak)
print("Mean (flat):", round(stats.mean(flat_peak), 2))

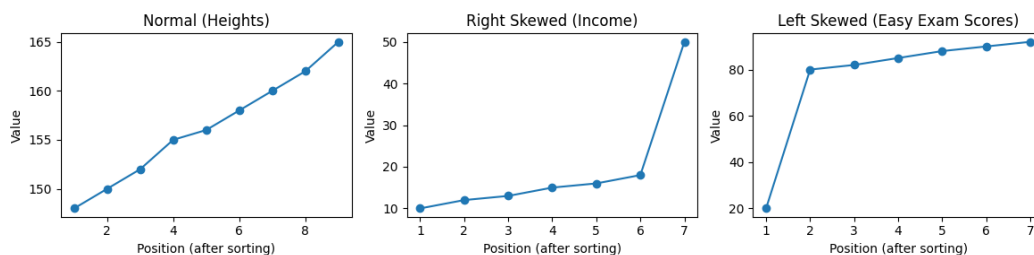
print("\nInterpretation:")
print("- sharp_peak → values crowded near 10 → sharper peak (higher kurtosis idea)")
print("- flat_peak → values spread out → flatter peak (lower kurtosis idea)")

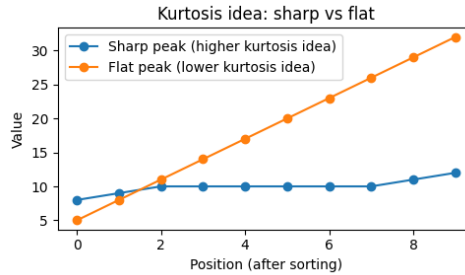
# 2) Line chart to *see* the difference
sharp_sorted = sorted(sharp_peak)
flat_sorted = sorted(flat_peak)
x_positions = list(range(1, len(sharp_sorted) + 1))

plt.plot(x_positions, sharp_sorted, marker="o", label="Sharp peak")
plt.plot(x_positions, flat_sorted, marker="o", label="Flat peak")

plt.title("Kurtosis idea: sharp vs flat")
plt.xlabel("Position (after sorting)")
plt.ylabel("Value")
plt.legend()
plt.tight_layout()
plt.show()
```

Output:





Summary

Exploratory Data Analysis (EDA) is the first step in data analysis where we look at the dataset, understand what it contains and clean it before building any model; it includes univariate analysis (studying one variable at a time using tables, graphs and simple statistics), bivariate analysis (studying the relationship between two variables using tools like scatter plots, correlation and cross tables) and multivariate analysis (studying three or more variables together using correlation matrices, pair plots, PCA and clustering to find important patterns and groups), along with handling missing data and outliers by detecting blanks or extreme values and deciding whether to fill, cap or remove them.

To do EDA correctly we must understand the type of data: quantitative data is numerical and can be discrete (whole numbers like number of students) or continuous (values in a range like height or temperature), while qualitative data is non-numerical and can be nominal (unordered categories like color or city) or ordinal (ordered categories like rating levels). It is also studied that the spread of data, which describes how values are scattered around the center; a normal distribution is a symmetric bell-shaped curve where mean, median and mode are equal, while skewed distributions are asymmetric with a longer tail to the right (positive skew) or left (negative skew). Finally, skewness measures this asymmetry and kurtosis measures how sharp the peak and tails are, helping us fully understand the shape and behavior of the data.